PILARES

FUNDAMENTALES

DE LA PROGRAMACION
ORIENTADA A OBEJTOS

INTRODUCCIÓN

¿QUÉ ES LA POO?

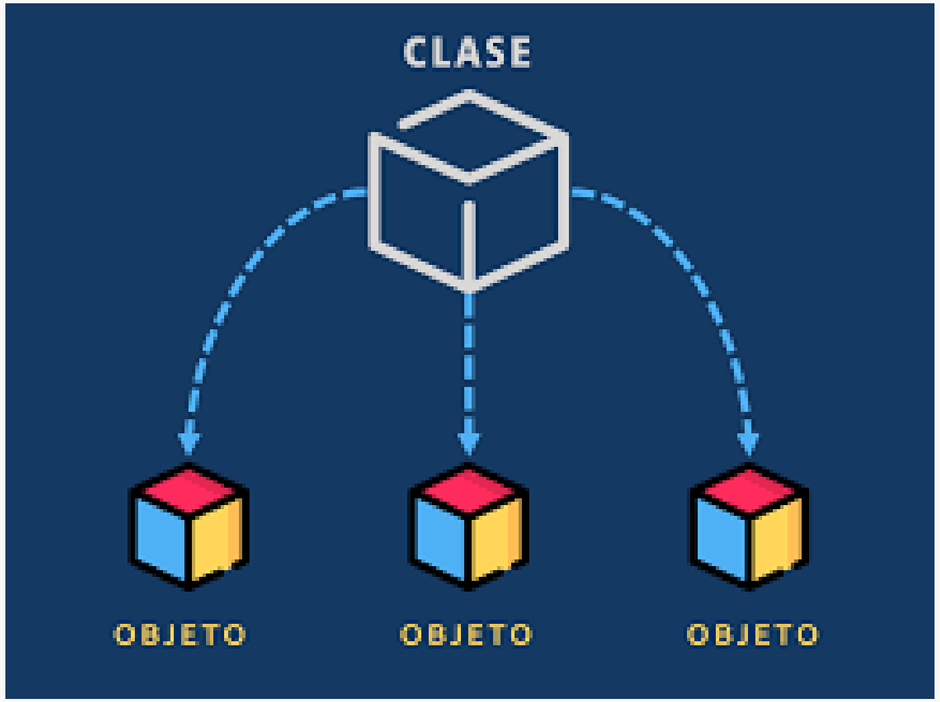
- La Programación Orientada a Objetos (POO) es un paradigma de programación que organiza el código en objetos, los cuales representan entidades del mundo real.

CADA OBJETO COMBINA:

- Atributos (datos)
- Métodos (funciones o comportamientos)

SU OBJETIVO PRINCIPAL ES FACILITAR EL DESARROLLO DE SOFTWARE MODULAR, REUTILIZABLE Y MANTENIBLE.

¿CUÁLES SON LOS PILARES FUNDAMENTALES DE LA POO?



Los 4 pilares fundamentales son:

- Encapsulamiento
- Abstracción
- Herencia
- Polimorfismo

DEFINICIÓN DE CADA PILAR

1. Encapsulamiento

```java
class Vehiculo {
    protected void obtenerInfoVehiculo() {
        System.out.println("Este es un vehículo.");
    }
}

class Automovil extends Vehiculo {
    public void mostrarInfoAutomovil() {
        obtenerInfoVehiculo();  // Llamada al método protegido de la superclase
    }
}

public class Ejemplo {
    public static void main(String[] args) {
        Automovil automovil = new Automovil();
        automovil.mostrarInfoAutomovil();
    }
}
```

Consiste en ocultar los detalles internos de un objeto y permitir el acceso solo a través de métodos definidos.

- Protege los datos
- Evita modificaciones indebidas

DEFINICIÓN DE CADA PILAR

2. Abstracción

```
abstract class ClassName {
    abstract void methodName();
    // Other methods and fields
}
```

Es la capacidad de mostrar solo lo esencial y ocultar lo complejo.

- Simplifica el uso del sistema
- Reduce la complejidad

DEFINICIÓN DE CADA PILAR

3. Herencia

```
class Superclass {
    // fields and methods
}

class Subclass extends Superclass {
    // additional fields and methods
}
```

Permite que una clase herede propiedades y métodos de otra.

- Reutilización de código
- Organización jerárquica

DEFINICIÓN DE CADA PILAR

4. Polimorfismo

```java
public class OverloadingExample {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        OverloadingExample obj = new OverloadingExample();
        System.out.println("Sum of integers: " + obj.add(5, 10));
        System.out.println("Sum of doubles: " + obj.add(5.5, 10.5));
    }
}
```

Permite que un mismo método tenga diferentes comportamientos según el contexto.

- Flexibilidad
- Código más dinámico

¿POR QUÉ LOS LENGUAJES ORIENTADOS A OBJETOS SOPORTAN LA POO?

LENGUAJES COMO JAVA, PYTHON O C++ INCLUYEN SOPORTE PARA POO PORQUE:

- Permiten modelar problemas reales fácilmente
- Facilitan el mantenimiento del código
- Promueven la reutilización
- Mejoran la escalabilidad de sistemas grandes

¿EN QUÉ CONTEXTOS APLICAR CADA PILAR?

ENCAPSULAMIENTO

- Sistemas financieros (seguridad de datos)
- Aplicaciones donde se controla el acceso a información

ABSTRACCIÓN

- Interfaces de usuario
- APIs
- Sistemas complejos (ocultar lógica interna)

HERENCIA

- Sistemas con jerarquías (ej: empleados, vehículos)
- Reutilización de código en proyectos grandes

POLIMORFISMO

- Interfaces gráficas
- Sistemas extensibles
- Manejo de múltiples tipos de objetos

MUCHAS GRACIAS